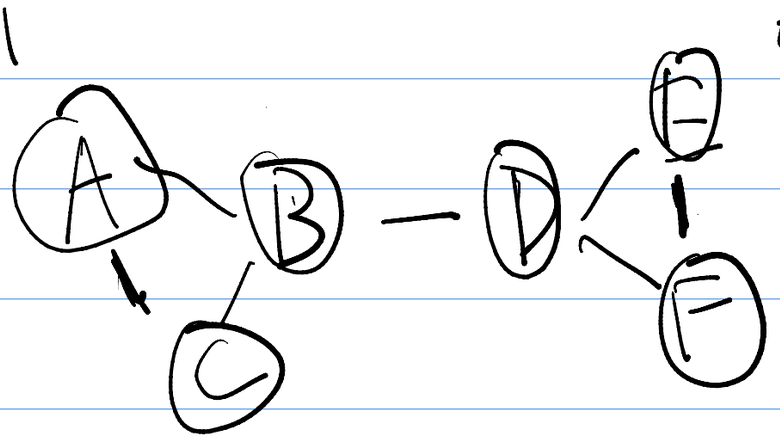
Network - ex 05

5.A.1



① $P_1 = \{A,B,C,D\} = AB, AC, B-C, C-D = 4$. $C(4,2) = 6$. Density $= 4/6$

$\{E,F\} = E-F$. Possible $C(2,2) = 1$. Density $= 1$

② Crossing edges $D-E, D-F = 2$. Possible crossing $= 2$ external density $= 2/9$

① $P_2 \{A,B,C\} = (A-B), (B-C), (A-C) = 3$, $C(3,2) = 3$. Density $= 1$

$\{D,E,F\} = (D-E), (D-F)(E-F) = 3$, $C(3,2) = 3$. Density $= 1$

② Crossing edges $= (B-D) = 1$. Possible $3 \times 3 = 9$. Ext density $= \frac{1}{9}$

③ P_2 achieves perfect internal density in both partition and minimal external density (0.11)

④ The graph consists of two complete triangles (A-B-C and D-E-F) joined by a single bridge edge B-D. the natural communities are $\{A,B,C\}$ and $\{D,E,F\}$

5.A.2

Modularity

Measures how strongly a network is partitioned into communities

$$Q = \frac{1}{2m} \sum \left[e_c - \frac{a_c^2}{4m} \right]$$

e_c = edges inside, a_c = Sum of degree, m = total edges

① P_2 : $\{A,B,C\}$ & $\{D,E,F\}$ $\deg(A) + \deg(B) + \deg(C) = 2+3+2 = 7$, $\deg(D) + \deg(E) + \deg(F) = 7$

$$Q = \frac{1}{2.7} 3 - \frac{(7)^2}{4.7} = \frac{1}{14} \times 3 - \frac{49}{28} = \frac{1}{14} \times \left[(3-1.75) + (3-1.75) \right] = \frac{1}{14} .25$$

② $Q = 0$ means Community Structure is no better than random

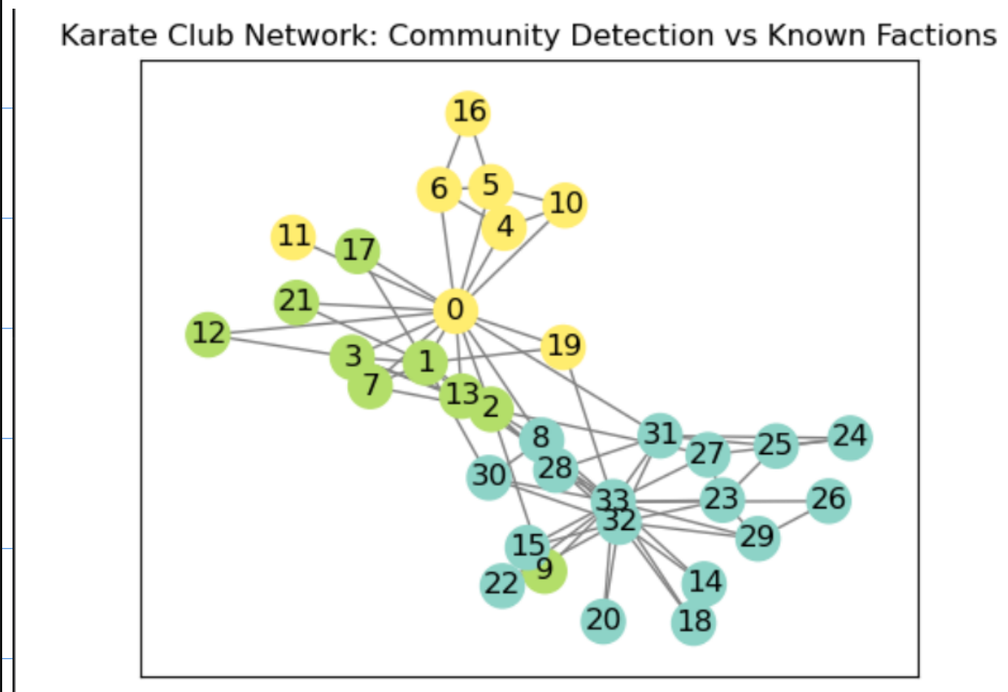
$Q = 1$ means perfect modularity

typical network $= 0.3 < Q < 0.7$

③ Shuffle, Q approaches 0. Any positive Q indicates more internal edges by chance

5.A.3

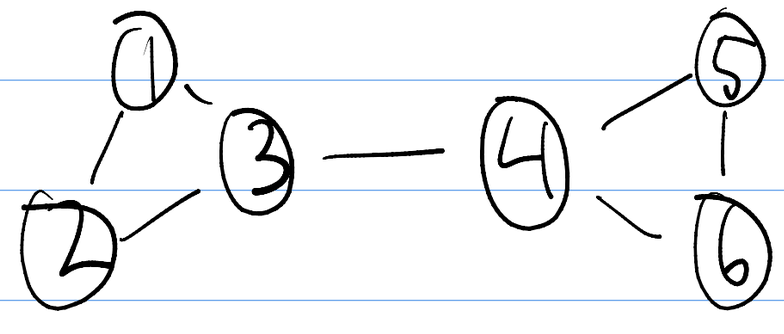
```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()

#greedy modularity communities
communities = nx.community.greedy_modularity_communities(G)

#compute modularity scores
modularity_score = nx.community.modularity(G, communities)
print(f"Modularity score: {modularity_score:.4f}")

#compare to the known faction labels. What fraction of nodes are correctly classified?
# Get the known faction labels from the graph
faction_labels = nx.get_node_attributes(G, 'club')
# Create a mapping from faction labels to community indices
faction_to_community = {}
for i, community in enumerate(communities):
    for node in community:
        faction_to_community[node] = i

# For each detected community, find the majority faction → assign as predicted label
community_predicted_faction = {}
for i, community in enumerate(communities):
    faction_counts = {}
    for node in community:
        f = faction_labels[node]
        faction_counts[f] = faction_counts.get(f, 0) + 1
    community_predicted_faction[i] = max(faction_counts, key=lambda k: faction_counts[k])

# Count how many nodes match their community's predicted faction
correct = 0
total = len(G.nodes())
```



```python
for node in G.nodes():
    if faction_labels[node] == community_predicted_faction[faction_to_community[node]]:
        correct += 1

print(f"Fraction of nodes correctly classified: {correct/total:.4f}")

# Visualize the communities
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
node_colors = [faction_to_community[node] for node in G.nodes()]
nx.draw_networkx(G, pos, with_labels=True, node_size=300, cmap=plt.cm.Set3,
                 node_color=node_colors, edge_color='gray')
plt.title("Karate Club Network: Community Detection vs Known Factions")
plt.show()
```

```
Modularity score: 0.4110
Fraction of nodes correctly classified: 0.9412
```

5.B.1



Edge betweeness

how often an edge lies on shortest paths between all pair of nodes

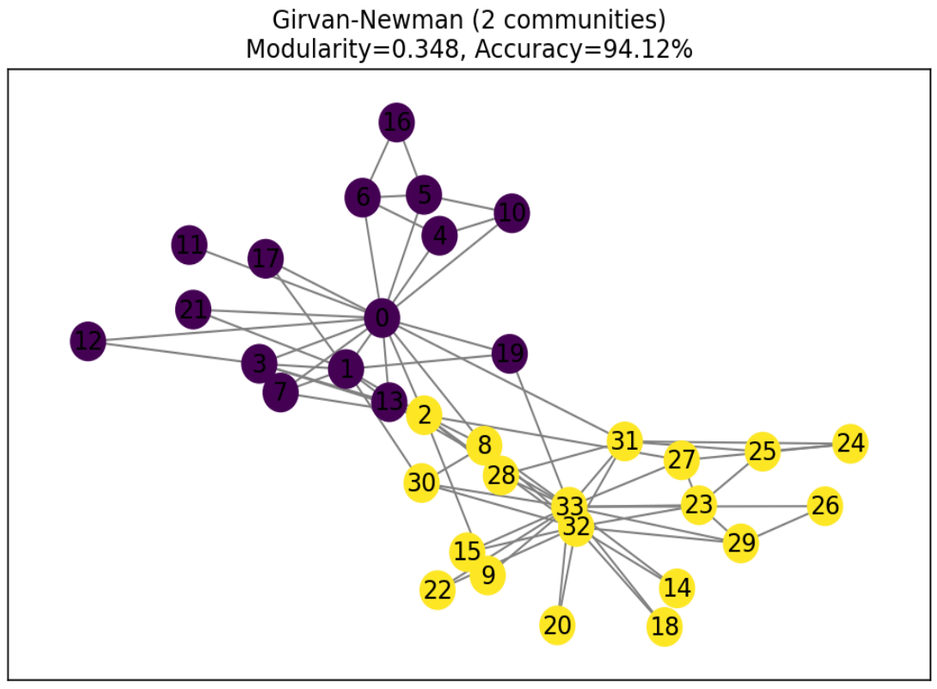
① 3-4 has the highest value since it lies on every path between nodes in {1,2,3} and {4,5,6} : 3×3 = 9 pairs. 9 betweness = 9 (unnormalised)

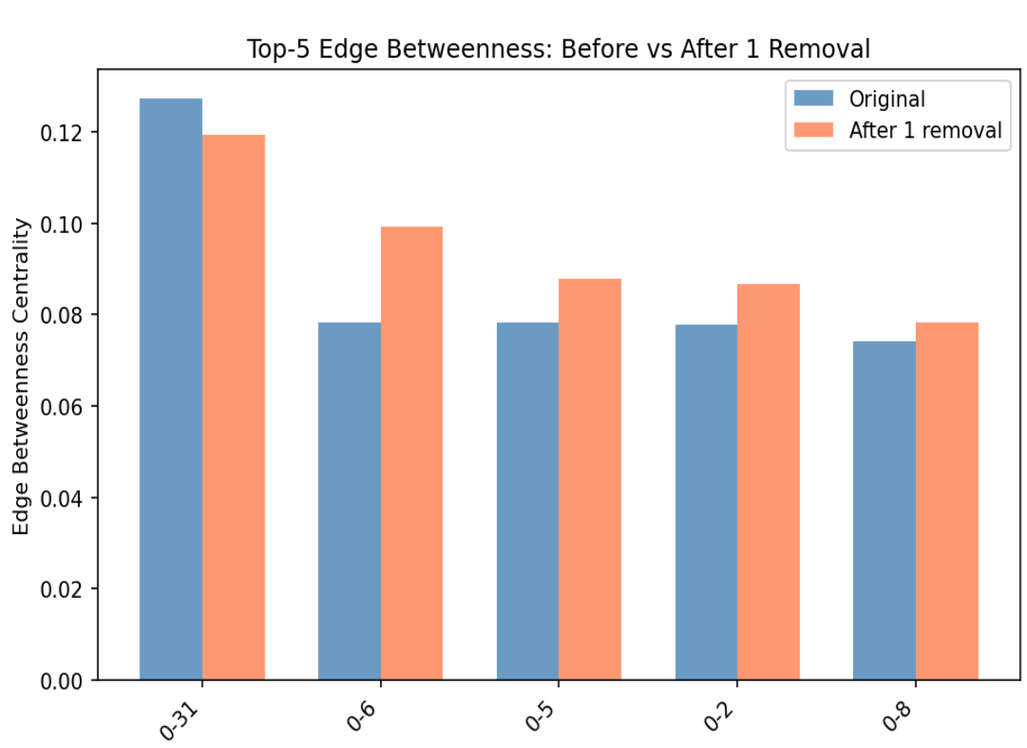
② after removing 3-4 the graph splits into two triangle

③ within each isolated triangle every edge would have the same betweness

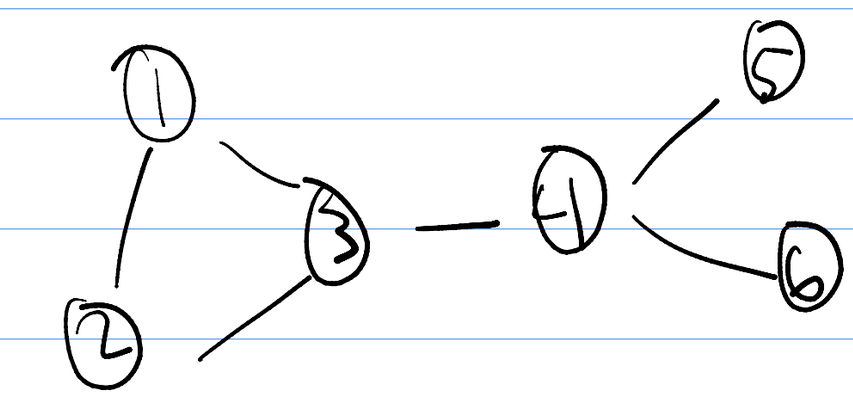
④ any path from {1,2,3} to {4,5,6} and vice versa would have to go through this edge. there is no alternative route.

5.B.2





5.C.1



Dendogram

a tree diagram showing the nested grouping from hierarchical clustering

① Height represents the distance between the two clusters being merged. Low height = very similar clusters ; high height = dissimilar clusters being forced together

② within each triangle nodes merge at low height first. the two triangles merge at the highest height because they're connected by a single bridge edge. So at height=1

③ Cut the dendogram at any height with $1 < h <$ inter-cluster height (ex: h=2 for complete/average)

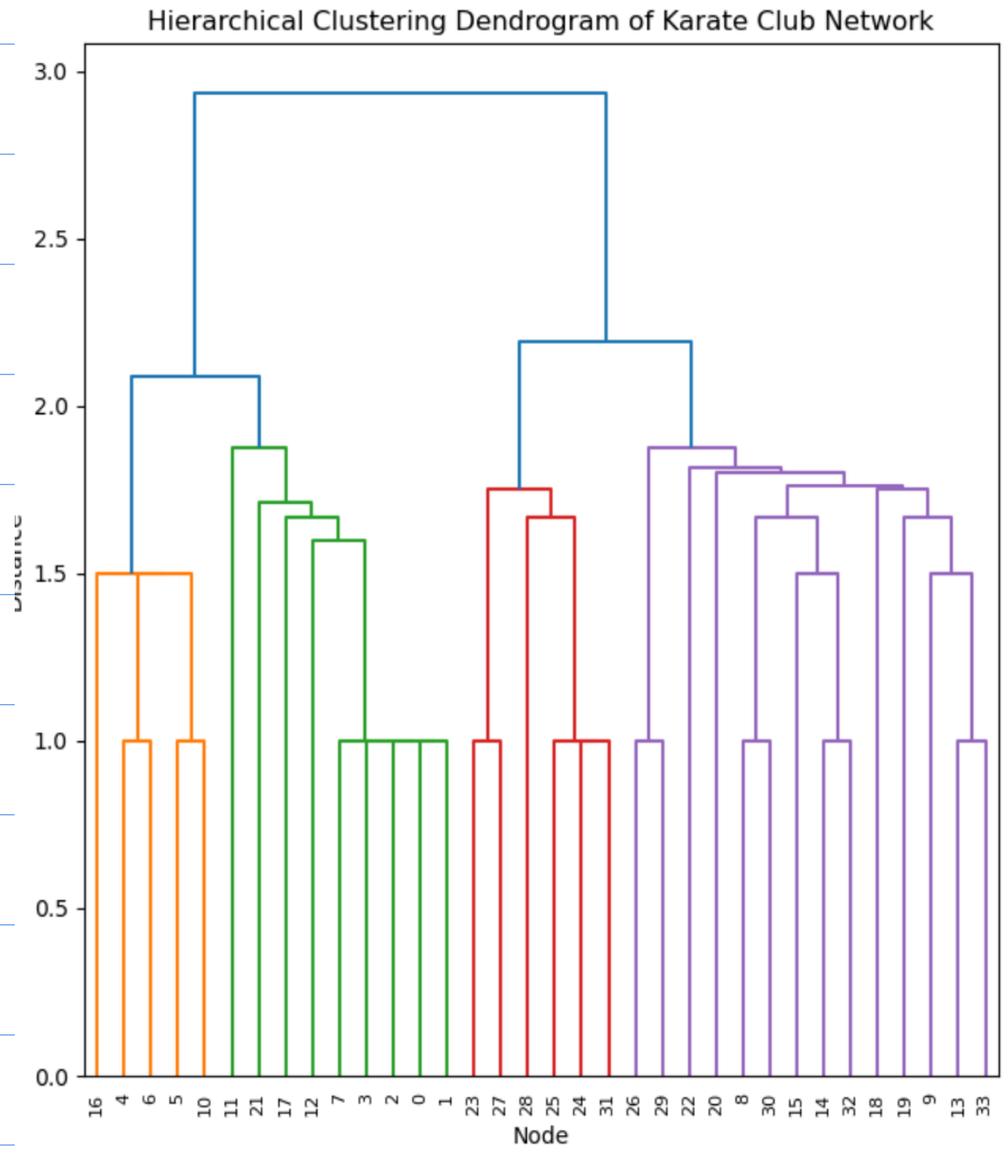
④ Direct-link similarity: two nodes are similar if they share an edge

Neighbourhood overlap similarity:

5.C.2

study > snippets > network-ex05 > 5.C.2.py > ...

```python
import scipy as sp
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np

#compute shortest path lengths between all pairs of nodes and use as distance matrix
G = nx.karate_club_graph()
shortest_paths = dict(nx.all_pairs_shortest_path_length(G))
n = len(G.nodes())
dist_matrix = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        dist_matrix[i, j] = shortest_paths[i][j]

from scipy.spatial.distance import squareform
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.stats import mode

# convert the square distance matrix to condensed form and apply average linkage
d_cond = squareform(dist_matrix)
Z = linkage(d_cond, method='average')

# plot the dendrogram
plt.figure(figsize=(10, 8))
_dn = dendrogram(Z, labels=list(G.nodes()), leaf_rotation=90)
plt.title("Hierarchical Clustering Dendrogram of Karate Club Network")
plt.xlabel("Node")
plt.ylabel("Distance")
plt.show()

# cut to obtain exactly 2 clusters
clusters = fcluster(Z, t=2, criterion='maxclust')

# evaluate fraction matching known factions
faction = nx.get_node_attributes(G, 'club')
true = np.array([0 if faction[i] == "Mr. Hi" else 1 for i in G.nodes()])
pred = clusters - 1  # make 0/1 labels
# map each predicted cluster to the majority true label
map_ = {}
for c in np.unique(pred):
    vals = true[pred == c]
    if vals.size == 0:
        map_[c] = 0
    else:
        counts = np.bincount(vals.astype(int))
        map_[c] = int(np.argmax(counts))
acc = (np.array([map_[c] for c in pred]) == true).mean()
print("Fraction matching known factions:", acc)
```




```
Fraction matching known factions: 0.9117647058823529
```